Простое освещение

Простое освещение

Тёмные подземелья, ночные уровни, фонарик в руке персонажа - освещение превращает плоскую 2D-сцену в атмосферную. И реализуется проще, чем кажется.

Зачем?

Освещение - это не только красиво. Это игровая **механика**, фича:

- **Стелс**: враг не видит игрока в темноте
- **Исследование**: факел освещает только часть подземелья - создаёт интригу. Особенно, если ничего не видно. И меню тоже.
- **Хоррор**: темнота сама по себе вызывает напряжение
- **Fog of war**: скрывает неизведанные области карты

Т.е. свет и тьма влияют на геймплей, а не только на картинку.

Идея: карта освещения

Как сделать так, чтобы часть экрана была тёмной, а часть - освещённой?

Решение в лоб: нарисовать полупрозрачный чёрный прямоугольник поверх всего. Но тогда **всё** одинаково тёмное - нет смысла.

Ок. Подход лучше: **карта освещения** (light map). Это отдельная поверхность размером с экран. На ней мы рисуем, **насколько освещён** каждый участок. Потом накладываем её поверх сцены **умножением** (multiply blend). Грубо говоря:

- Белый пиксель на карте -> соответствующий пиксель сцены видна полностью
- Чёрный пиксель -> пиксель сцены полностью тёмный
- Серый -> полутень

Python C#

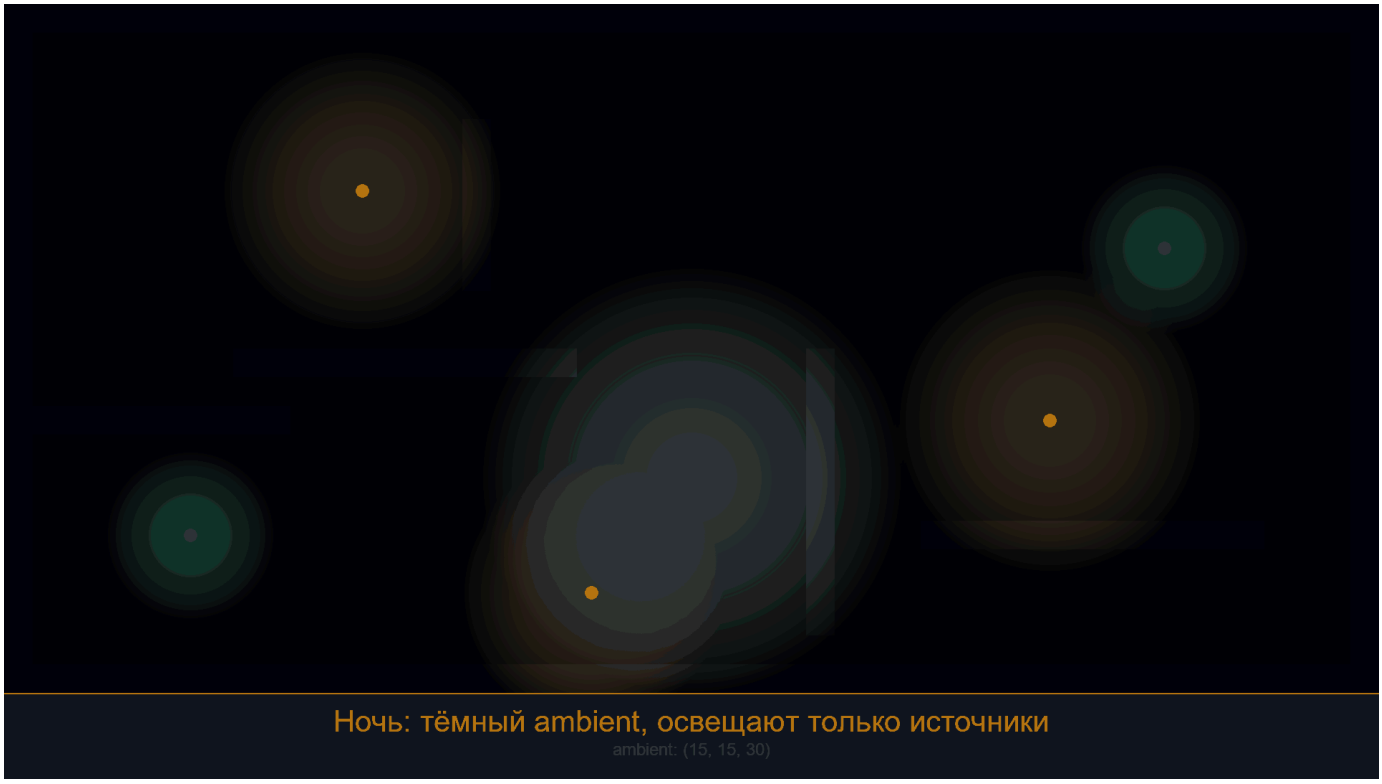
```python
# create light map (once)
light_map = pygame.Surface((SCREEN_W, SCREEN_H))

# each frame:
light_map.fill((20, 20, 30))  # ambient: almost dark, slight blue tint
```

```
# draw circle of light around player (screen coordinates!)
pygame.draw.circle(light_map, (255, 255, 200),
                   camera.apply(player.x, player.y), 150)

# draw scene as usual
screen.blit(background, (0, 0))
draw_entities(screen)

# apply light map on top (multiply blend)
screen.blit(light_map, (0, 0), special_flags=pygame.BLEND_RGB_MULT)
```

`ambient` - это цвет "без света". `(20, 20, 30)` даёт почти чёрную ночь с лёгким синим оттенком. `(100, 100, 110)` - пасмурные сумерки. `(255, 255, 255)` - обычный день.

Порядок отрисовки: сначала сцена, потом light map поверх. Если рисовать наоборот - ничего не сработает. Multiply blend **затемняет** то, что уже на экране.

Позиции источников света - **экранные координаты**, не мировые. Если есть камера с прокруткой, учитывайте. Подробнее - Камера.



Мягкий свет: градиент

Резкий круг выглядит неестественно - свет обрывается неожиданно. Нам нужен **радиальный градиент**: яркий в центре, плавно затухающий к краям.

Принцип: для каждого пикселя текстуры считаем расстояние до центра, нормализуем в диапазон `0..1`, и получаем **интенсивность** = `1 - dist`. Возведение в степень управляет формой спада:

- `falloff = 1.5` - на середине радиуса (`dist/radius = 0.5`): `0.5^1.5 = 0.35`. Свет быстро гаснет - **компактное пятно**
- `falloff = 0.8` - на середине радиуса: `0.5^0.8 = 0.57`. Свет держится дольше - **размытое зарево**

```
intensity = max(0, 1 - dist/radius)
intensity = intensity ^ falloff   // falloff > 1 = sharper, < 1 = softer
pixel_color = light_color * intensity
```

В pygame это делается через `Surface` + `set_at()` попиксельно (медленно, но нужно один раз при загрузке). В MonoGame - либо `Texture2D.SetData()`, либо проще подготовить PNG в графическом редакторе: белый круг с плавным спадом на чёрном фоне, а цвет задать при отрисовке через `Color`.

И тут важно: на light map мы рисуем с **аддитивным** блендингом (`BLEND_RGB_ADD` / `BlendState.Additive`). Это значит, что свет от нескольких источников **складывается** - два факела рядом освещают ярче, чем один.

Python | C#

```python
# additive: each blit adds brightness to the light map
light_map.blit(player_light, (px - radius, py - radius),
               special_flags=pygame.BLEND_RGB_ADD)
light_map.blit(torch_light, (tx - r, ty - r),
               special_flags=pygame.BLEND_RGB_ADD)
```

Текстуру градиента создаём **один раз** при загрузке. Попиксельная генерация - операция не из быстрых, каждый кадр её вызывать не стоит.

Несколько источников света

Карта освещения хороша тем, что на неё можно рисовать **сколько угодно** источников. Факелы на стенах, костёр, светящиеся грибы - тот же код, другие параметры.

Тот же принцип, что и в системах частиц или ИИ врагов: одинаковый код, разные данные. Хотим мерцающий факел? Меняем радиус каждый кадр на случайную величину:

Python C#

```python
torch_radius = base_radius + random.randint(-5, 5)
```

Хотим не круговое мерцание? Добавляем анимацию нашей окружности.

Смена дня и ночи

Допустим, в игре есть цикл день/ночь. Днём всё видно, вечером темнеет, ночью - только искусственные источники света. Реализуется через ambient-цвет карты освещения.

Нужна **линейная интерполяция** (lerp) между опорными точками: день -> вечер -> ночь -> рассвет. Время суток - число от `0.0` до `1.0`, опорные цвета - через какие ambient-значения проходит цикл:

Python C#

```python
# key points: (time_of_day, ambient_color)
ambient_keys = [
    (0.0,  (15, 15, 35)),   # midnight
    (0.25, (60, 40, 50)),   # dawn
    (0.4,  (255, 255, 255)),# day
    (0.75, (60, 40, 30)),   # dusk
    (1.0,  (15, 15, 35)),   # midnight again
]
# lerp between nearest two keys based on current time_of_day
# time_of_day += speed * dt; time_of_day %= 1.0
```

Фишка в том, что искусственные источники рисуются **всегда**. Днём ambient и так `(255, 255, 255)` - добавление света ничего не меняет. Но вечером ambient падает, и факелы, фонари, окна домов становятся видны. Ночью они - единственное, что освещает сцену.

Хотим тёплый вечер с фонарями? Ambient `(60, 40, 30)` + оранжевые источники. Холодная лунная ночь? Ambient `(15, 15, 35)` + один большой бледно-голубой "лунный свет" на всю сцену.

Тени от стен

Хм. До сих пор наш свет проходил сквозь стены - факел в соседней комнате светит через стену. Чтобы это исправить, нужен **рейкастинг**: бросаем лучи от источника света во все стороны и проверяем, не упёрлись ли они в стену.

Идея та же, что и луч видимости у врагов - только вместо одного луча "глаза -> игрок" мы бросаем лучи веером из источника света.

```
для каждого угла от 0° до 360°:
    бросаем луч из позиции источника
    двигаемся по лучу шаг за шагом
    если уперлись в стену -> остановились
    все пиксели до стены - освещены
    все пиксели за стеной - в тени
```

Результат рейкастинга - **полигон видимости** (visibility polygon): область, которую "видит" источник. Рисуем этот полигон на light map с градиентом - и свет больше не проходит сквозь стены.

Более интересные темы для исследования: **visibility polygon**, **2D shadow casting**, **shadow volumes**. Для тайловых карт есть упрощённый вариант - пометить каждый тайл как "непрозрачный" и не рисовать свет за ним.

Цветной свет

Всё, что описано выше, работает с цветом. Градиентная текстура хранит не просто яркость, а RGB. Факел - `(255, 180, 60)`, магический кристалл - `(100, 150, 255)`, ядовитое болото - `(80, 255, 50)`. При аддитивном блендинге цвета смешиваются: оранжевый факел рядом с синим кристаллом даст белёсое пятно на стыке.

Цвет - мощный инструмент для подсказок игроку. Красный свет из-за двери? Опасность. Зелёный? Может, магазин. Тёплый жёлтый? Безопасная зона. Грубо говоря, свет **общается** с игроком без единого слова текста.

Итого

Вся система освещения - это одна дополнительная поверхность (light map) и пара строк блендинга. Порядок действий:

1. Создать light map размером с экран
2. Каждый кадр: залить ambient-цветом
3. Нарисовать градиентные текстуры света (additive blend)
4. Нарисовать сцену как обычно
5. Наложить light map поверх (multiply blend)

Что	Где подробнее
deltaTime для анимаций и дня/ночи	Основной цикл
Лучи видимости, рейкастинг	ИИ врагов
Тайлы и структура уровня	Уровни и поиск путей
Объекты как данные (свет = позиция + радиус + цвет)	ECS, частицы
Генерация подземелий, где освещение критично	Процедурная генерация
Камера и экранные координаты	Камера

Powered by Wiki.js